

Golf Round Tracker

Developer reference / repository file outline

This reference maps project-owned files to their responsibilities, interfaces, and maintenance concerns. It describes the current implementation, not an end-user tutorial.

Architecture

The frontend is a vanilla HTML/CSS/JavaScript PWA. GitHub Pages serves static assets; JavaScript handles scoring, history, statistics, and localStorage. Python does not run on the hosted site.

The optional local runtime serves the same frontend through FastAPI. Pydantic validates completed rounds, SQLite persists them, and Python calculates statistics. Browser code remains responsible for UI and device integrations.

```
index.html + style.css
  api-client.js -> script.js -> nearby-courses.js
                        |           |
                    localStorage  external APIs
                        |
                    optional /api
                        |
                    backend/app.py
                      /       \
                models.py    statistics.py
                      \
                    database.py -> SQLite
```

Contents

- Page 2: frontend entry points and controller workflows.
- Page 3: API client and external search integrations.
- Page 4: Python HTTP routes and domain models.
- Page 5: database, statistics, and persistence contracts.
- Page 6: PWA files, launchers, and configuration.
- Page 7: tests, documentation, generated files, and maintenance.
- Pages 8-14: all 84 named top-level script.js functions in source order.

Third-party library internals, Git objects, and generated bytecode are outside this outline. Search source identifiers rather than relying on line numbers.

Frontend files

index.html and style.css

index.html defines home/setup, custom-course, active-round, history, and statistics screens, forms, dialogs, and controls. IDs are the interface with JavaScript DOM lookups. Script order is api-client.js, script.js, then nearby-courses.js; these are classic scripts sharing selected globals, not ES modules.

style.css owns typography, layout, cards, forms, scorecard tables, dialogs, responsive rules, and mobile safe-area spacing. The .hidden class is part of the screen-switching contract. Keep IDs, labels, control constraints, and CSS selectors aligned when changing markup.

script.js

The main controller contains DOM references, event registration, built-in courses, active-round state, browser persistence, history, dashboard rendering, offline statistics, weather capture, and optional Python coordination. It also registers the service worker.

Core state: selectedCourse, currentHoleIndex, roundResults, roundConditions, activeRoundToken, editingCourseId, and editingCompletedRoundId.

Course path: loadSavedCourses hydrates stored cards; searchCourses/loadCourseDetails/selectApiTee transform provider data. Custom-course handlers validate and store cards. updateRoundFormatOptions/getRoundSelection resolve the actual hole list. Hon-E-Kor has three named nines and ordered pairings; tee color and nine selection are separate.

Round path: startRound initializes state; displayCurrentHole renders it; saveHoleAndContinue validates and advances. Draft handlers persist input, support return-home/resume, and remove drafts on discard or completion. finishRound aggregates the scorecard and saves completed history. Putts and penalties are analysis fields already included in score, not additional strokes added to it.

History path: showEditCompletedRound creates an edit dialog; saveEditedCompletedRound validates rows, recomputes totals, preserves completion date, adds editedAt, and replaces the matching ID. Deletion removes one record after confirmation.

Statistics path: getCareerHistory applies the cutoff. renderStatistics requests Python preview calculations or falls back locally and guards against stale results. renderHomeDashboard calculates locally. JavaScript and Python formulas must remain consistent.

Weather path: captureRoundConditions uses course or device coordinates and Open-Meteo. activeRoundToken rejects late responses for a different round. This is a snapshot, not continuous tracking. The appendix lists individual functions.

Browser integration files

api-client.js

An immediately invoked function exposes `window.golfPythonApi`. The root is `window.GOLF_API_URL` when supplied; otherwise `/api` is enabled for localhost or 127.0.0.1 with a nonempty port. A static local server can match this heuristic without providing Python; controller error handling supplies browser fallback.

request sends JSON with a five-second timeout, rejects non-success HTTP status codes, and handles empty 204 responses.

- `getRounds()`: GET `/rounds`; retrieves completed history.
- `replaceRounds(rounds)`: PUT `/rounds`; replaces all server history.
- `getStatistics()`: GET `/statistics`; available database-summary helper.
- `calculateStatistics(rounds)`: POST `/statistics/preview`; calculates the supplied snapshot without saving it. This is the statistics screen's path.

A remote override alone is not a deployment solution. Authentication, CORS, TLS, and synchronization would need separate design.

nearby-courses.js

Encapsulates ZIP and device-location handlers and reuses `script.js` result rendering and scorecard loading. `readJson` checks HTTP status with a 15-second timeout.

ZIP submission validates five digits, resolves coordinates with `Zippopotam.us`, and invokes `searchNearby`. Device search wraps `getCurrentPosition` in a Promise with a 12-second timeout and permission/error handling.

`searchNearby` disables competing searches, requests up to 50 `OpenGolfAPI` matches, filters U.S. states, sorts by distance, and displays results. Supported radii are 10, 25, and 50 miles. ZIP distances start at the ZIP center; distances are not driving routes. This file does not deliberately persist search coordinates in `localStorage`.

External dependency boundaries

`OpenGolfAPI` supplies course search and available scorecards. `Zippopotam.us` receives ZIP queries. `Open-Meteo` receives coordinates for weather. Provider coverage and response availability are outside the app's control; saved-course scoring must remain independent.

Nearby location is a one-time fix, not distance-to-green, hazard mapping, or shot tracking. These features and authenticated cloud synchronization are not implemented.

Python HTTP and models

backend/app.py

Creates FastAPI and RoundDatabase. Imports CompletedRound, round_summary, and dashboard_statistics. GOLF_TRACKER_DATABASE overrides backend/data/golf_tracker.db.

- GET /api/health -> health_check: health status JSON.
- GET /api/rounds -> list_rounds: completed rounds, newest first.
- POST /api/rounds -> create_round: validate/upsert; return round, status 201.
- PUT /api/rounds/{round_id} -> update_round: validate/upsert; mismatched URL/body IDs return 400.
- DELETE /api/rounds/{round_id} -> delete_round: 204 on removal; 404 if absent.
- PUT /api/rounds -> replace_rounds: validate list, atomically replace history, return saved count.
- GET /api/statistics -> get_statistics: round_summary over database history.
- POST /api/statistics/preview -> preview_statistics: dashboard_statistics over supplied rounds, without persistence.

The browser uses whole-history replacement for saves, edits, and deletions. Individual mutation routes exist but are not its primary save path. POST and single-record PUT use upsert semantics.

home serves index.html; public_file allows only PUBLIC_FILES; /icons is mounted separately. These FastAPI rules exclude backend source, database, and .git, but do not describe GitHub Pages hosting behavior. The API has no authentication or multi-user isolation; the launcher binds locally to 127.0.0.1.

backend/models.py

ApiModel accepts Python names and JSON aliases and preserves extra fields. HoleResult defines positive hole/par/score values, nonnegative putts/penalties, optional yardage, and nullable fairway text.

CompletedRound defines identity, date, course/tee/layout metadata, conditions, ratings, totals, and 1-18 holes. calculate_totals runs after validation and recomputes totals. as_browser_json emits camelCase aliases such as totalScore.

Limits: dates are strings; fairway is not an enum; unique hole numbers and exactly 9/18 holes are not required. Totals remain required inputs although recalculation overwrites them. This is data validation, not a complete golf rules engine.

Storage and calculations

backend/database.py

RoundDatabase uses SQLite: rounds has id primary key, completed_at text for ordering, and round_json for the entire scorecard. _connect opens named-row connections; _create_tables initializes schema. list_rounds decodes records; save_round upserts; delete_round reports existence; replace_rounds replaces history transactionally.

Bound SQL parameters separate values from statements. Connections close in finally blocks; failed replacement rolls back. Hole analysis happens in Python rather than normalized SQL. There is no migration system or multi-client conflict resolution.

backend/statistics.py

round_summary averages raw round totals for a simple database summary. dashboard_statistics groups by hole count, accumulates score/putt/penalty totals and hole types, computes fairway percentage from total opportunities, and selects best round by strokes-over-par per hole with total-score tie-breaking.

estimated_differential uses $(113 / \text{slope}) * (\text{score} - \text{rating})$ for rated 18-hole rounds, otherwise an 18-hole-scaled par approximation. handicap_estimate uses up to 20 recent rounds, a sample-size selection/adjustment table, one-decimal rounding, and a cap of 54. Fewer than three rounds yields no estimate. This is not an official Handicap Index implementation.

Browser storage contracts

- golfTrackerActiveRound: draft course, hole index/results, conditions, token.
- golfTrackerRoundHistory: completed snapshots.
- golfTrackerSavedCourses / golfTrackerFavoriteCourses: saved cards / favorite IDs.
- golfTrackerCareerStart / golfTrackerPreviousCareerStart: cutoff / one-level undo.
- golfTrackerPythonPending: outstanding history-save marker.

saveRoundHistory writes locally first, then queues ordered whole-history API saves. The pending flag clears only when the saved snapshot still matches current history. Failed requests retain pending state for retry.

At startup, pending browser history is retried. Otherwise an empty server may be seeded locally; nonempty server history replaces browser history. This is not a merge. SQLite stores completed rounds only, not drafts, courses, favorites, or career settings. Storage is origin-specific; phone/desktop and different ports do not share it automatically.

PWA and project configuration

service-worker.js

Caches the app shell at install, removes old caches at activation, and uses network-first handling for eligible same-origin GET requests. Navigation can fall back to index.html. /api and external services are excluded from app-shell caching. CACHE_NAME is currently v9.

Keep the cached asset list and cache version aligned with frontend releases. Cache storage holds application files; localStorage holds application data.

manifest.webmanifest and icons/

The manifest declares app name, start URL, scope, colors, standalone display, orientation, and icon paths. Relative paths support subdirectory hosting. icons/icon-180.png is the Apple touch icon; icons/icon-192.png and icons/icon-512.png provide manifest installation assets. These are static assets, not executable modules.

run.py and Start Golf Tracker.cmd

run.py resolves project paths, parses --port/--test, prepares .venv, checks dependencies, installs missing requirements, and invokes Uvicorn or unittest. Start Golf Tracker.cmd changes to the project directory and selects .venv Python, py, or python. pause retains console output after exit.

Commands: python run.py starts the app; python run.py --port 8001 changes the port; python run.py --test runs tests. With dependencies installed, uvicorn backend.app:app --reload supports local development.

Supporting files

- backend/requirements.txt: FastAPI, Pydantic, and Uvicorn version ranges, not a pinned lockfile.
- backend/__init__.py: backend package marker and description.
- backend/tests/__init__.py: test package marker and description.
- .editorconfig: UTF-8, four-space indentation, LF defaults, whitespace rules, and CRLF for .cmd files.
- .gitignore: excludes environments, Python/test caches, and backend/data; it does not untrack files already committed.

Tests, docs, and maintenance

Test files

backend/tests/test_statistics.py: make_even_par_round fixture; differential and initial-adjustment checks, summaries, empty dashboard, round-length groups, empty-card rejection, and stale-total recalculation.

backend/tests/test_database.py: sample_round fixture and temporary databases; save/list, replacement, existing/missing deletion, and rollback after duplicate IDs trigger replacement failure.

backend/tests/test_api.py: ApiTests starts Uvicorn with a temporary database and available local port; HTTP checks verify non-persisting previews, homepage serving, and private-path denial. Cleanup stops the process. The suite contains 13 tests; it does not automate browser or device GPS coverage.

Documentation and generated files

- README.md: feature overview, setup, and documentation links.
- docs/CODE_GUIDE.md: short architecture walkthrough.
- docs/COMPLETE_GUIDE.md: editable source for this reference; page delimiters are part of its build contract.
- tools/build_guide.py: ReportLab renderer and curated function-description map. Extracts top-level script.js function names, checks coverage, appends the index, and validates page count/title placement.
- output/pdf/golf-tracker-explained.pdf: generated reference. Rebuild with python tools/build_guide.py using reportlab and pypdf, which are documentation-only dependencies.

backend/data/golf_tracker.db is runtime data. .venv holds dependencies, __pycache__ holds generated bytecode, .git holds version metadata, and tmp/pdfs is temporary document QA output. These are not application source files.

Change-impact checklist

New round field: update HTML/control wiring, draft save/restore, completed snapshots, Python aliases/validation, edit/history rendering, and fixtures. Formula changes require both JavaScript fallback and Python updates. New frontend assets require cache-list/version and public-file allowlist checks.

Keep deletion distinct from career filtering. Home best, statistics best-round, and course-specific best metrics use different comparisons. Known gaps include backup/export, hosted authentication, multi-device conflict handling, and automated browser/service-worker upgrade tests.

Function index 1

script.js - search these exact names in your editor

createHonEKorCourse()

Build all three nines using the yardages for the selected tee color.

getSavedCourses()

Read saved course JSON; keep valid cards and exclude obsolete Hon-E-Kor imports.

getFavoriteCourseIds()

Read the stored list of favorite IDs and discard invalid entries.

isFavoriteCourse()

Answer whether a course ID is in that favorite list.

toggleFavoriteCourse()

Add or remove the selected favorite; refresh its label and home shortcuts.

isValidCourse()

Check a course object's required fields and hole data before trusting it.

loadSavedCourses()

Load stored cards into the courses object and course dropdown.

saveCourse()

Insert or replace a saved course by ID, then update the dropdown.

addCourseOption()

Create or refresh one dropdown option for a course.

getCourseOptionLabel()

Build a readable label with favorite mark, name, and tee.

selectSavedCourse()

Select the matching course and update layout choices and summary.

updateRoundFormatOptions()

Offer valid full-round, front/back-nine, or ordered multi-nine choices.

Function index 2

script.js - search these exact names in your editor

getRoundSelection()

Return the actual hole list and label for the chosen layout.

updateSelectedCourseCard()

Show selected course, tee, layout, total par, and yardage.

updateCourseActionButtons()

Enable favorite controls and show editing only for supported courses.

searchCourses()

Submit name text to the course service, filter U.S. results, and handle failure.

setSearchLoading()

Disable name, ZIP, and location search buttons during an active search.

renderCourseSearchResults()

Build clickable results with name, city/state, and distance when supplied.

loadCourseDetails()

Fetch a result's scorecard and tees; route Hon-E-Kor to built-in verified layouts.

loadHonEKorDetails()

Prepare the three tee choices and multi-nine options for Hon-E-Kor.

getTeeLabel()

Build a readable tee label from the provider's tee metadata.

selectApiTee()

Translate API tee/hole data into an app course and save the selected card.

editSelectedCourse()

Find the selected saved card and open its editing form.

showCustomCourse()

Open the add/edit screen and populate inputs when editing an existing card.

Function index 3

script.js - search these exact names in your editor

renderCustomHoleRows()

Create editable hole rows matching the selected hole count.

saveCustomCourse()

Validate custom course inputs, construct the card, and store it.

changeNumberInput()

Apply a plus/minus change without going below the allowed minimum.

saveActiveRound()

Serialize the active course, hole index, scores, weather, and token locally.

startRound()

Confirm draft replacement, initialize the chosen holes, and begin scoring/weather lookup.

displayCurrentHole()

Fill the scoring controls and progress display for the current hole.

saveHoleAndContinue()

Validate entered values, save the hole, and advance or finish the round.

saveCurrentHoleWithoutValidation()

Capture usable draft input values without the full next-hole validation step.

goToPreviousHole()

Preserve current inputs, move back one hole, and redraw the form.

saveInputProgress()

Save changed inputs and refresh the current score while a round is active.

checkForSavedRound()

Show or hide resume controls based on a valid stored draft.

getValidSavedRound()

Parse and validate draft JSON and restore a usable course snapshot.

Function index 4

script.js - search these exact names in your editor

resumeSavedRound()

Restore active-round state and display its saved hole.

saveRoundAndGoHome()

Preserve current entries and the draft before returning to Home.

discardSavedRound()

Confirm removal of the active draft and clear the resume UI.

showScorecardOverview()

Save current inputs and open an overview of the active round's holes.

captureRoundConditions()

Request a weather snapshot and ignore results belonging to an older round token.

getCourseCoordinates()

Use known course coordinates or ask for device location as a fallback.

renderRoundConditions()

Display the captured weather in the active-round screen.

getCardinalDirection()

Convert wind degrees into a compass label such as N or SW.

getWeatherLabel()

Convert the weather service's numeric condition code to readable words.

formatConditions()

Produce the short temperature and wind label used on history cards.

getRoundHistory()

Read completed history JSON and filter invalid records.

getCareerStart()

Return a valid stored career date, or null for all history.

Function index 5

script.js - search these exact names in your editor

getCareerHistory()

Keep completed rounds on or after the career start date.

resetCareerStatistics()

Require two confirmations and RESET; save a new start date and previous period.

undoCareerReset()

Confirm and restore the previous career period, then remove the undo marker.

isValidCompletedRound()

Check that a history object has the fields and numeric hole values the UI needs.

saveCompletedRound()

Prepend a finished card to history and call the shared save function.

saveRoundHistory()

Save locally first; in Python mode queue an ordered database snapshot save.

syncRoundsWithPythonBackend()

On startup retry pending data, seed an empty server, or load server history.

deleteCompletedRound()

Find the requested ID, ask for confirmation, remove it, and redraw.

showEditCompletedRound()

Build a temporary per-hole edit dialog for the selected completed card.

createEditNumberInput()

Create an accessible numeric input with a label and minimum value.

saveEditedCompletedRound()

Validate edited rows, recalculate totals, add editedAt, and save the replacement.

calculateCompletedRoundTotals()

Add scores, par, putts, penalties, and fairways for the edited hole list.

Function index 6

script.js - search these exact names in your editor

showRoundHistory()

Switch the visible screen to history and render its cards.

showSetup()

Return Home and refresh course actions, resume status, and dashboard.

showStatistics()

Switch to the statistics screen and begin rendering its current career.

renderStatistics()

Ask Python for grouped statistics when enabled; fall back locally and reject stale results.

renderHomeDashboard()

Show local career summary, latest all-history round, and favorite shortcuts.

formatDateOnly()

Format a valid date in the user's locale without its time.

calculateHandicapEstimate()

Offline version of the recent-differential selection and initial adjustment table.

calculateEstimatedDifferential()

Offline version of the rated-18 formula or par-based approximation.

renderPersonalBests()

Group records by course, tee, and layout; display the lowest score in each group.

calculateStatistics()

Accumulate local averages, round-length groups, hole-type totals, and best round.

renderHoleTypeStatistics()

Display average score and score-to-par for par-3, par-4, and par-5 holes.

renderRecentResults()

Show up to five results; compare oldest/newest only when their lengths match.

Function index 7

script.js - search these exact names in your editor

formatAverage()

Render a numeric average with one decimal place.

formatGroupedAverage()

Divide each length group's selected total by rounds and add 9H/18H labels.

formatAverageToPar()

Round a score-to-par average and display Even or a signed number.

renderRoundHistory()

Clear and rebuild the history list, count, and empty-state message.

createHistoryCard()

Build one course/date/summary card with scorecard, edit, and delete controls.

createStat()

Build a small label/value element for a summary statistic.

createScorecard()

Build the hole table with scores, putts, fairways, and penalties.

formatRoundDate()

Display a completed round's date and time in the user's locale.

formatScoreToPar()

Display zero as Even, positive scores with +, and negative scores normally.

formatFairway()

Translate stored fairway codes into readable display text.

updateCurrentScore()

Add completed/entered hole scores and compare them with the corresponding par.

finishRound()

Build and save a complete scorecard, remove the draft, and show the final summary.